大数据应用案例

费子成　　苏州大学

E-mail: feizicheng@suda.edu.cn

个人主页：https://web.suda.edu.cn/feizicheng/





01 Spark编程实践

Spark编程实践

Spark安装

启动Spark Shell

RDD基本操作

Spark应用程序

Spark安装

安装Spark之前需要安装Java环境和Hadoop环境。
Spark下载地址： http://spark.apache.org

登录Linux系统（本教程统一采用hadoop用户登录），打开浏览器，访问Spark官网（https://archive.apache.org/dist/spark/spark-3.4.0/）

在页面中选择下载“spark-3.4.0-bin-without-hadoop.tgz”。假设下载后的文件被保存在“~/Downloads”目录下。

Spark安装

解压安装包spark-3.4.0-bin-without-hadoop.tgz至路径 /usr/local：

```
$ sudo tar -zxf ~/Downloads/spark-3.4.0-bin-without-hadoop.tgz -C /usr/local/
$ cd /usr/local
$ sudo mv ./spark-3.4.0-bin-without-hadoop/ ./spark   # 更改文件夹名
$ sudo chown -R hadoop ./spark                        # 此处的 hadoop 为系统用户名
```

配置Spark 的Classpath。

配置Spark 的Classpath。

编辑该配置文件，在文件最后面加上如下一行内容：

```
export SPARK_DIST_CLASSPATH=$(/usr/local/hadoop/bin/hadoop classpath)
```

保存配置文件后，就可以启动、运行Spark了。Spark包含多种运行模式：**单机模式、伪分布式模式、完全分布式模式**。本章使用单机模式运行Spark。若需要使用HDFS中的文件，则在使用Spark前需要启动Hadoop。

启动Spark Shell

01

Spark Shell 提供了简单的方式来学习 Spark API。

02

Spark Shell可以以实时、交互的方式来分析数据。

03

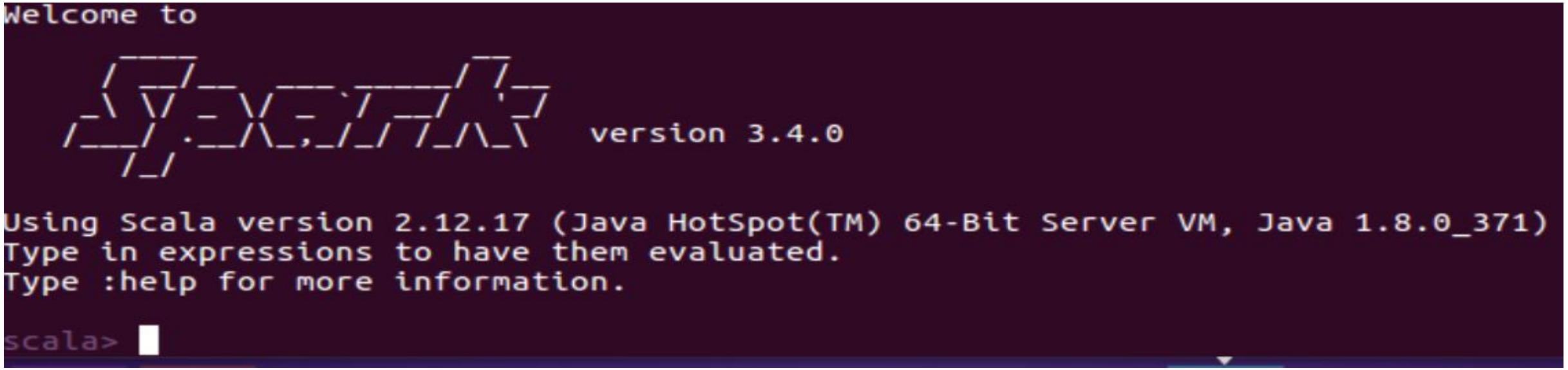
Spark Shell支持Scala和Python。

本章节内容选择使用Scala进行编程实践，了解Scala有助于更好地掌握Spark。

执行如下命令启动Spark Shell：

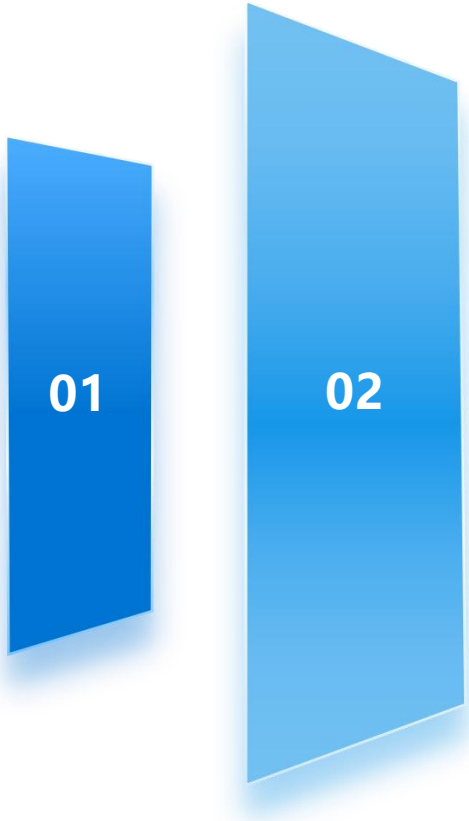
```
$ ./bin/spark-shell
```

启动Spark Shell成功后在输出信息的末尾可以看到“Scala >”的命令提示符，如下图所示。



RDD基本操作

Spark的主要操作对象是RDD，RDD可以通过多种方式灵活创建，既可通过导入外部数据源建立（如位于本地或HDFS中的数据文件），也可从其他的RDD转化而来。



可以对RDD执行各种操作来完成所需的业务处理逻辑。RDD操作包括两种类型，即转换（Transformation）操作和行动（Action）操作。

RDD基本操作

■ 转换操作

对于RDD而言，每一次转换操作都会产生不同的RDD，供给下一个操作使用。RDD的转换过程是惰性求值的，也就是说，整个转换过程只是记录了转换的轨迹，并不会发生真正的计算，只有遇到行动操作时，才会触发“从头到尾”的真正的计算。表中给出了常用的RDD转换操作API，其中很多操作都是高阶函数，比如，filter(func)就是一个高阶函数，这个函数的输入参数func也是一个函数。

操作	含义
filter(func)	筛选出满足函数func的元素，并返回一个新的数据集
map(func)	将每个元素传递到函数func中，并将结果返回为一个新的数据集
flatMap(func)	与map()相似，但每个输入元素都可以映射到0或多个输出结果
groupByKey()	应用于(K,V)键值对的数据集时，返回一个新的(K, Iterable)形式的数据集
reduceByKey(func)	应用于(K,V)键值对的数据集时，返回一个新的(K, V)形式的数据集，其中每个值是将每个key传递到函数func中进行聚合后的结果

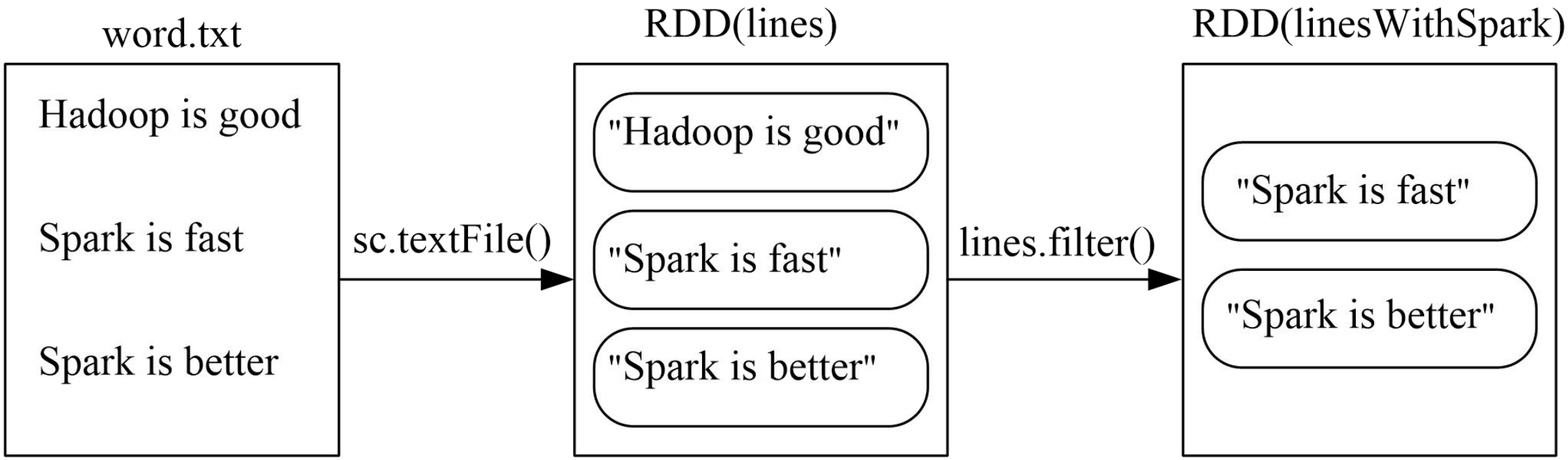
常用的RDD转换操作API

RDD基本操作

- filter(func)

filter(func)操作会筛选出满足函数func的元素，并返回一个新的数据集。例如：

```
scala>  val  lines = sc.textFile("file:///usr/local/spark/mycode/rdd/word.txt")
scala>  val  linesWithSpark=lines.filter(line => line.contains("Spark"))
```

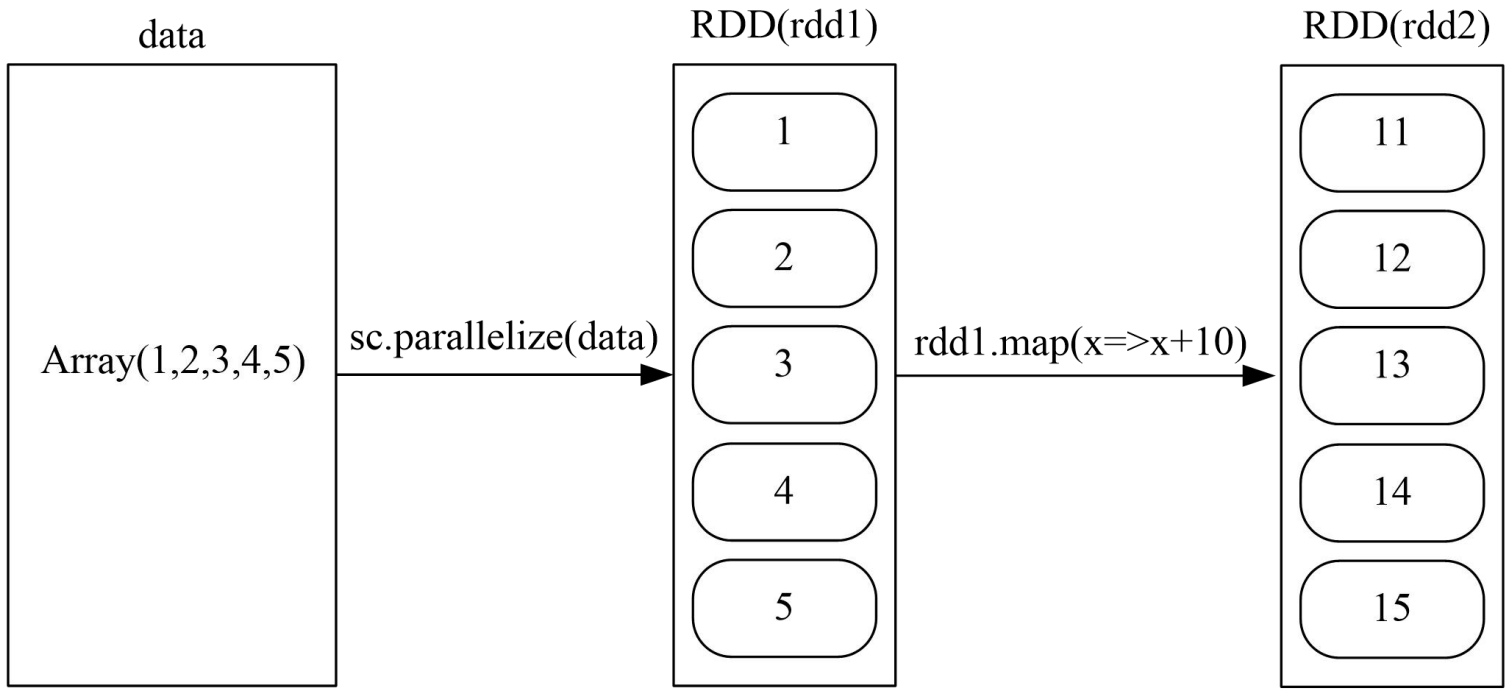


RDD基本操作

- map(func)

map(func)操作将每个元素传递到函数func中，并将结果返回为一个新的数据集。例如：

```
scala> data=Array(1,2,3,4,5)
scala> val  rdd1= sc.parallelize(data)
scala> val  rdd2=rdd1.map(x=>x+10)
```

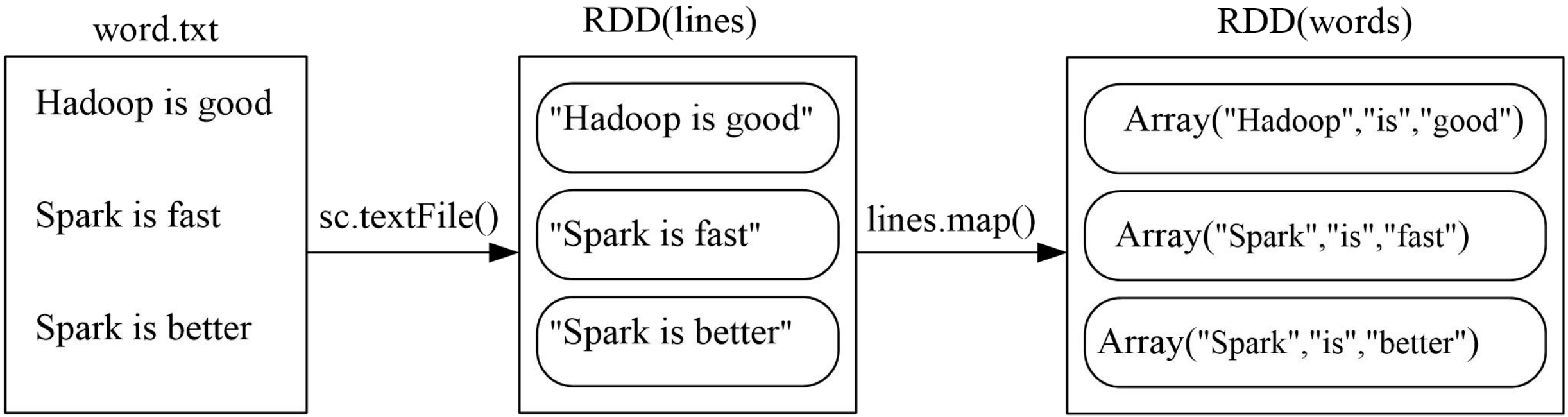


RDD基本操作

- map(func)

下面是另外一个实例：

```
scala> val  lines = sc.textFile("file:///usr/local/spark/mycode/rdd/word.txt")
scala> val  words=lines.map(line => line.split(" "))
```

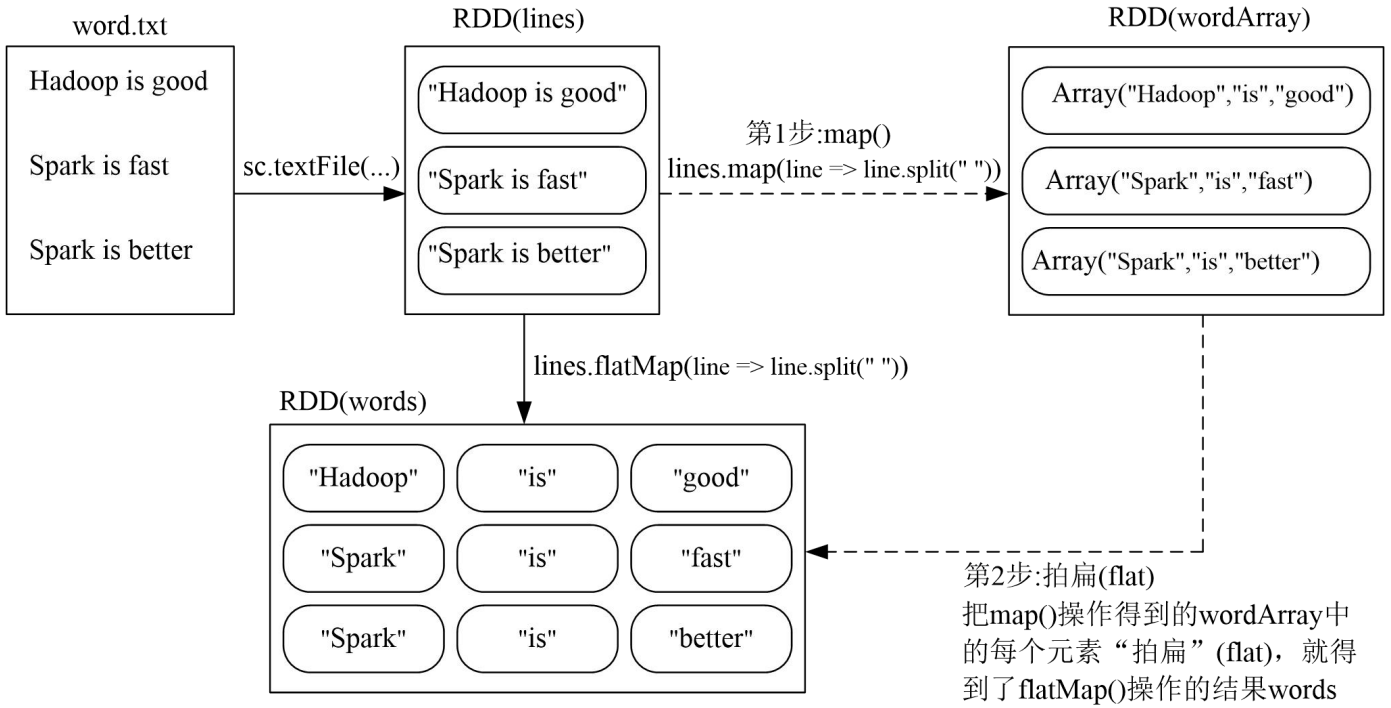


RDD基本操作

- flatMap(func)

flatMap(func)与map()相似，但每个输入元素都可以映射到0或多个输出结果。例如：

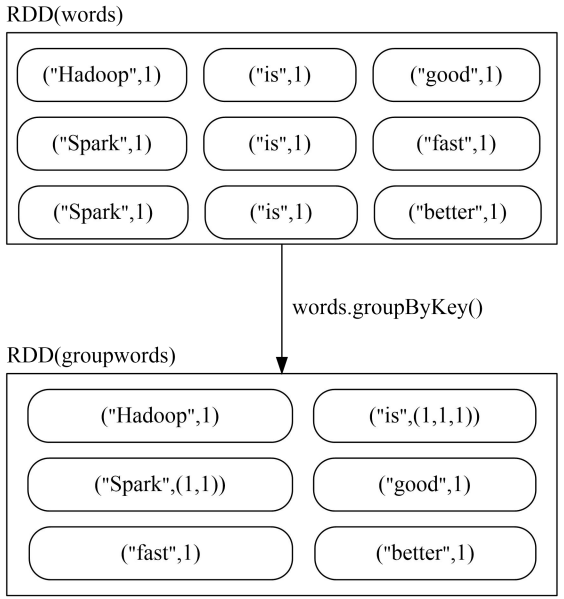
```
scala> val  lines = sc.textFile("file:///usr/local/spark/mycode/rdd/word.txt")
scala> val  words=lines.flatMap(line => line.split(" "))
```



RDD基本操作

- groupByKey()

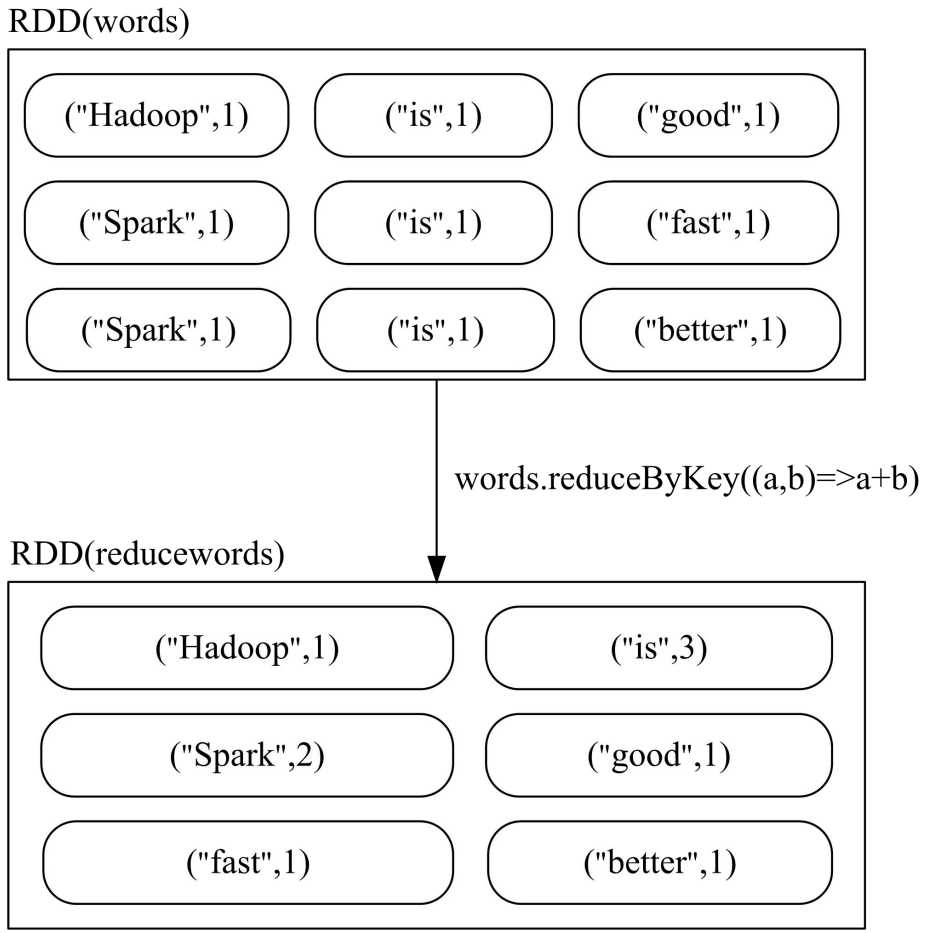
groupByKey()应用于(K,V)键值对的数据集时，返回一个新的(K, Iterable)形式的数据集。如图所示，名称为words的RDD中包含了9个元素，每个元素都是<String,Int>类型，也就是(K,V)键值对类型。words. groupByKey()操作执行以后，所有key相同的键值对，它们的value都被归并到一起。例如，("is",1)、("is",1)、("is",1)这3个键值对的key相同，就会被归并成一个新的键值对("is",(1,1,1))，其中，key是"is"，value是(1,1,1)，而且，value会被封装成Iterable（一种可迭代集合）。



RDD基本操作

- **reduceByKey(func)**

reduceByKey(func)应用于(K,V)键值对的数据集时，返回一个新的(K, V)形式的数据集，其中的每个值是将每个key传递到函数func中进行聚合后得到的结果。



RDD基本操作

■ 行动操作

行动操作是真正触发计算的地方。Spark程序只有执行到行动操作时，才会执行真正的计算，从文件中加载数据，完成一次又一次转换操作，最终，完成行动操作得到结果。下表列出了常用的RDD行动操作API。

操作	含义
count()	返回数据集中的元素个数
collect()	以数组的形式返回数据集中的所有元素
first()	返回数据集中的第一个元素
take(n)	以数组的形式返回数据集中的前*n*个元素
reduce(func)	通过函数func（输入两个参数并返回一个值）聚合数据集中的元素
foreach(func)	将数据集中的每个元素传递到函数func中运行

常用的RDD行动操作API

RDD基本操作

■ 行动操作

行动操作是真正触发计算的地方。Spark程序只有执行到行动操作时，才会执行真正的计算，从文件中加载数据，完成一次又一次转换操作，最终，完成行动操作得到结果。下表列出了常用的RDD行动操作API。

```
scala> val  rdd=sc.parallelize(Array(1,2,3,4,5))
rdd: org.apache.spark.rdd.RDD[Int]=ParallelCollectionRDD[1] at parallelize at <console>:24
scala> rdd.count()
res0: Long = 5
scala> rdd.first()
res1: Int = 1
scala> rdd.take(3)
res2: Array[Int] = Array(1,2,3)
scala> rdd.reduce((a,b)=>a+b)
res3: Int = 15
scala> rdd.collect()
res4: Array[Int] = Array(1,2,3,4,5)
scala> rdd.foreach(elem=>println(elem))
1
2
3
4
5
```

Spark应用程序

- 用sbt编译打包Scala程序

sbt（Simple Build Tool）是对Scala或Java进行编译的一个工具，类似于Maven或Ant，需要JDK 1.8或更高版本的支持，并且可以在Windows和Linux两种操作系统中安装使用。sbt需要下载安装，可以访问“http://www.scala-sbt.org”下载安装文件sbt-1.9.0.tgz，保存到下载目录。假设下载目录为“~/Downloads”，安装目录为“/usr/local/sbt”，则执行如下命令将下载后的文件复制至安装目录中：

```
$ sudo mkdir /usr/local/sbt                    # 创建安装目录
$ cd ~/Downloads
$ sudo tar -zxvf ./sbt-1.9.0.tgz -C /usr/local
$ cd /usr/local/sbt
$ sudo chown -R hadoop /usr/local/sbt          # 此处的hadoop为系统当前用户名
$ cp ./bin/sbt-launch.jar ./  #把bin目录下的sbt-launch.jar复制到sbt安装目录下
```

Spark应用程序

- 用sbt编译打包Scala程序

接着在安装目录中使用下面命令创建一个Shell脚本文件，用于启动sbt：

```
$ vim /usr/local/sbt/sbt
```

该脚本文件中的代码如下：

```
#!/bin/bash
SBT_OPTS="-Xms512M -Xmx1536M -Xss1M -XX:+CMSClassUnloadingEnabled -XX:MaxPermSize=256M"
java $SBT_OPTS -jar 'dirname $0'/sbt-launch.jar "$@"
```

保存后，还需要为该Shell脚本文件增加可执行权限：

```
$ chmod u+x /usr/local/sbt/sbt
```

Spark应用程序

- 用sbt编译打包Scala程序

然后，可以使用如下命令查看sbt版本信息：

```
$ cd /usr/local/sbt
$ ./sbt sbtVersion
Java HotSpot(TM) 64-Bit Server VM warning: ignoring option MaxPermSize=256M; support was removed in 8.0
[warn] No sbt.version set in project/build.properties, base directory: /usr/local/sbt
[info] welcome to sbt 1.9.0 (Oracle Corporation Java 1.8.0_371)
[info] set current project to sbt (in build file:/usr/local/sbt/)
[info] 1.9.0
```

Spark应用程序

- 用sbt编译打包Scala程序

现在，就可以使用“/usr/local/sbt/sbt package”的命令来打包Scala编写的Spark程序了。
下面以一个简单的程序为例，介绍如何打包并运行Spark程序。该程序的功能是统计文本文件中包含字母a和字母b的各有多少行。首先执行如下命令创建程序根目录，并创建程序所需的文件夹结构：

```
$ mkdir ~/sparkapp                          # 创建程序根目录
$ mkdir -p ~/sparkapp/src/main/scala        # 创建程序所需的文件夹结构
```

Spark应用程序

- 用sbt编译打包Scala程序

接着使用下面命令创建一个SimpleApp.scala文件：

```
$ vim ~/sparkapp/src/main/scala/SimpleApp.scala
```

该文件是程序的代码内容，具体代码如下：

```
import org.apache.spark.SparkContext
import org.apache.spark.SparkContext._
import org.apache.spark.SparkConf

object SimpleApp {
  def main(args: Array[String]) {
    val logFile = "file:///usr/local/spark/README.md" // 用于统计的文本文件
    val conf = new SparkConf().setAppName("Simple Application")
    val sc = new SparkContext(conf)
    val logData = sc.textFile(logFile, 2).cache()
    val numAs = logData.filter(line => line.contains("a")).count()
    val numBs = logData.filter(line => line.contains("b")).count()
    println("Lines with a: %s, Lines with b: %s".format(numAs, numBs))
  }
}
```

Spark应用程序

- 用sbt编译打包Scala程序

然后使用下面命令创建一个simple.sbt文件：

```
$ vim ~/sparkapp/simple.sbt
```

该文件用于声明该应用程序的信息以及与Spark的依赖关系，具体内容如下：

```
name := "Simple Project"
version := "1.0"
scalaVersion := "2.12.17"
libraryDependencies += "org.apache.spark" %% "spark-core" % "3.4.0"
```

需要说明的是，上面的scalaVersion表示Scala版本。

Spark应用程序

- 用sbt编译打包Scala程序

最后，执行如下命令使用sbt进行打包：

```
$ cd ~/sparkapp
$ /usr/local/sbt/sbt package
```

打包成功后，会输出“success”的提示，如下所示：

```
$~/sparkapp$ /usr/local/sbt/sbt package
Java HotSpot(TM) 64-Bit Server VM warning: ignoring option MaxPermSize=256M; support was removed in 8.0
[info] welcome to sbt 1.9.0 (Oracle Corporation Java 1.8.0_371)
[info] loading project definition from /home/hadoop/sparkapp/project
[info] loading settings for project sparkapp from simple.sbt ...
[info] set current project to Simple Project (in build file:/home/hadoop/sparkapp/)
[success] Total time: 5 s, completed Oct 18, 2023 12:51:03 AM
```

Spark应用程序

- 用sbt编译打包Scala程序

有了最终生成的JAR包后，通过spark-submit就可以提交到Spark中运行了，命令如下：

```
$ /usr/local/spark/bin/spark-submit --class "SimpleApp" ~/sparkapp/target/scala-2.12/simple-project_2.12-1.0.jar
```

该应用程序的执行结果如下：

```
Lines with a: 72, Lines with b: 39
```

Spark应用程序

■ 用Maven编译打包Java程序

Maven是对Java进行编译的一个工具，需要下载安装，可以访问https://maven.apache.org下载。假设本地的下载目录为“~/Downloads”，本地的安装目录为“/usr/local/maven”，下载得到的安装文件为apache-maven-3.9.2-bin.zip，需要执行如下命令，将下载后的文件复制至安装目录中：

```
$ sudo unzip ~/Downloads/apache-maven-3.9.2-bin.zip -d /usr/local
$ cd /usr/local
$ sudo mv apache-maven-3.9.2/ ./maven
$ sudo chown -R hadoop ./maven
```

在终端执行如下命令创建一个文件夹sparkapp2作为应用程序根目录：

```
$ cd ~ #进入用户主文件夹
$ mkdir -p ./sparkapp2/src/main/java
```

在“./sparkapp2/src/main/java”目录下使用下面命令建立一个名为

```
SimpleApp.java的文件：
$ vim ./sparkapp2/src/main/java/SimpleApp.java
```

Spark应用程序

- 用Maven编译打包Java程序

在SimpleApp.java这个文件中添加如下代码：

```
/*** SimpleApp.java ***/
import org.apache.spark.api.java.*;
import org.apache.spark.api.java.function.Function;
import org.apache.spark.SparkConf;

public class SimpleApp {
  public static void main(String[] args) {
    String logFile = "file:///usr/local/spark/README.md"; // Should be some file on your system
    SparkConf conf=new SparkConf().setMaster("local").setAppName("SimpleApp");
    JavaSparkContext sc=new JavaSparkContext(conf);
    JavaRDD<String> logData = sc.textFile(logFile).cache();

    long numAs = logData.filter(new Function<String, Boolean>() {
      public Boolean call(String s) { return s.contains("a"); }
    }).count();

    long numBs = logData.filter(new Function<String, Boolean>() {
      public Boolean call(String s) { return s.contains("b"); }
    }).count();

    System.out.println("Lines with a: " + numAs + ", lines with b: " + numBs);
  }
}
```

Spark应用程序

- 用Maven编译打包Java程序

该程序依赖Spark Java API，因此我们需要通过Maven进行编译打包。再使用下面命令在“./sparkapp2”目录中新建文件pom.xml：

```
$ vim ./sparkapp2/pom.xml
```

在pom.xml文件中添加如下代码，声明该独立应用程序的信息以及与Spark的依赖关系：

```
<project>
  <groupId>cn.edu.xmu</groupId>
  <artifactId>simple-project</artifactId>
  <modelVersion>4.0.0</modelVersion>
  <name>Simple Project</name>
  <packaging>jar</packaging>
  <version>1.0</version>
  <repositories>
    <repository>
      <id>jboss</id>
      <name>JBoss Repository</name>
      <url>http://repository.jboss.com/maven2/</url>
    </repository>
  </repositories>
  <dependencies>
    <dependency> <!-- Spark dependency -->
      <groupId>org.apache.spark</groupId>
      <artifactId>spark-core_2.12</artifactId>
      <version>3.4.0</version>
    </dependency>
  </dependencies>
</project>
```

Spark应用程序

- 用Maven编译打包Java程序

为了保证Maven能够正常运行，先执行如下命令检查整个应用程序的文件结构：

```
$ cd ~/sparkapp2
$ find .
```

应用程序的文件结构如下右图所示。
接着，可以通过如下代码将整个应用程序打包成JAR包（注意：电脑需要保持连接网络的状态，而且由于是首次运行，同样需要下载依赖包，因此这个过程会消耗几分钟的时间）：

```
$ /usr/local/maven/bin/mvn package
```

若出现如下右图所示信息，则说明JAR包生成成功。

```
.
./src
./src/main
./src/main/java
./src/main/java/SimpleApp.java
./pom.xml
```

```
[INFO] ------------------------------------------------------------------------
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------------------------------------
[INFO] Total time:  5.655 s
[INFO] Finished at: 2023-06-18T00:54:52-07:00
[INFO] ------------------------------------------------------------------------
```

Spark应用程序

- 用Maven编译打包Java程序

最后，需要将生成的JAR包通过spark-submit提交到Spark中运行，命令如下：

```
$ /usr/local/spark/bin/spark-submit --class "SimpleApp" ~/sparkapp2/target/simple- project-1.0.jar 2>&1 | grep "Lines with a"
```

最后得到的结果如下：

```
Lines with a: 72, Lines with b:  39
```